

MSA with Genetic Algorithms

BioInf 2025/2026 1st Semester

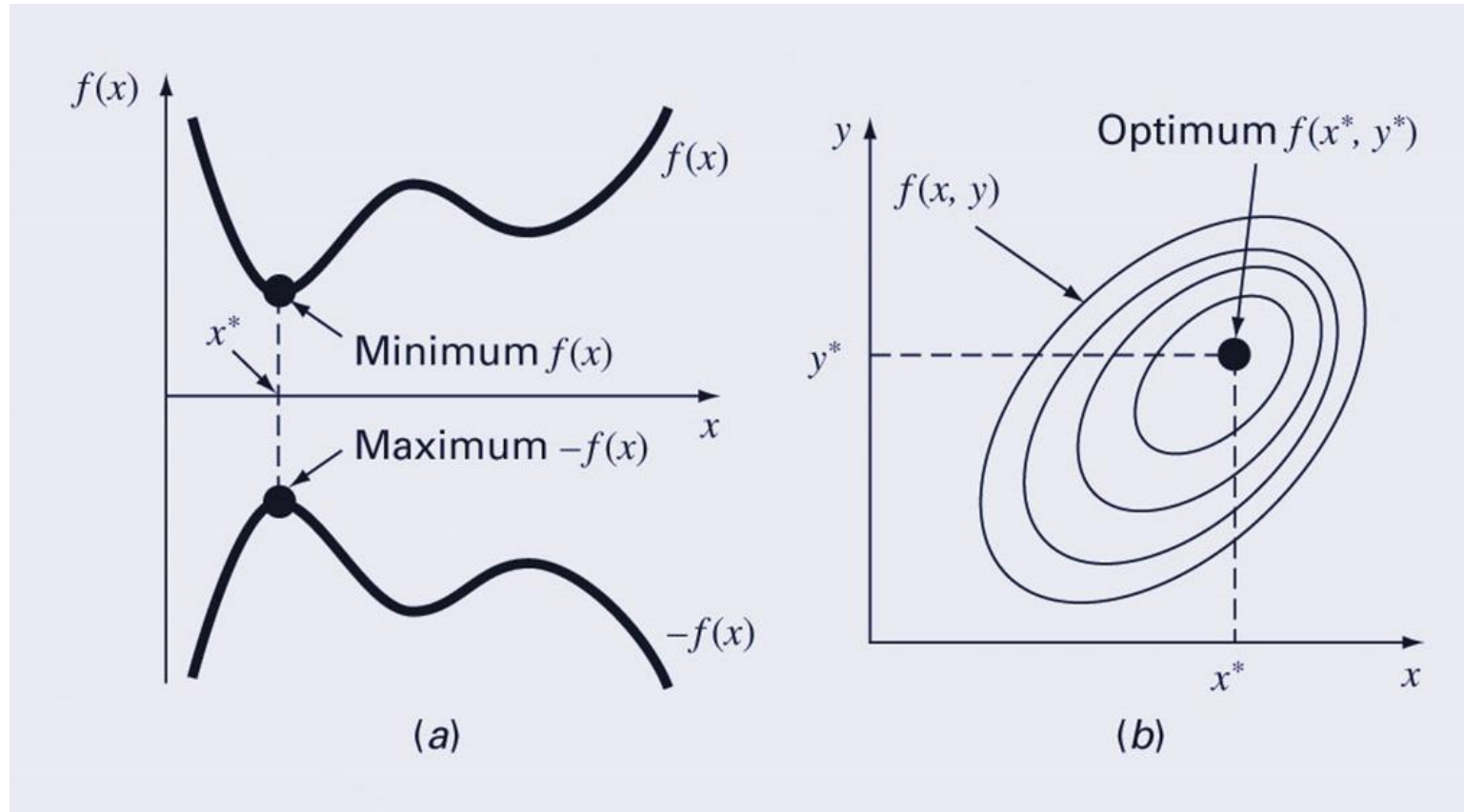
André Lamúrias

a.lamurias@fct.unl.pt

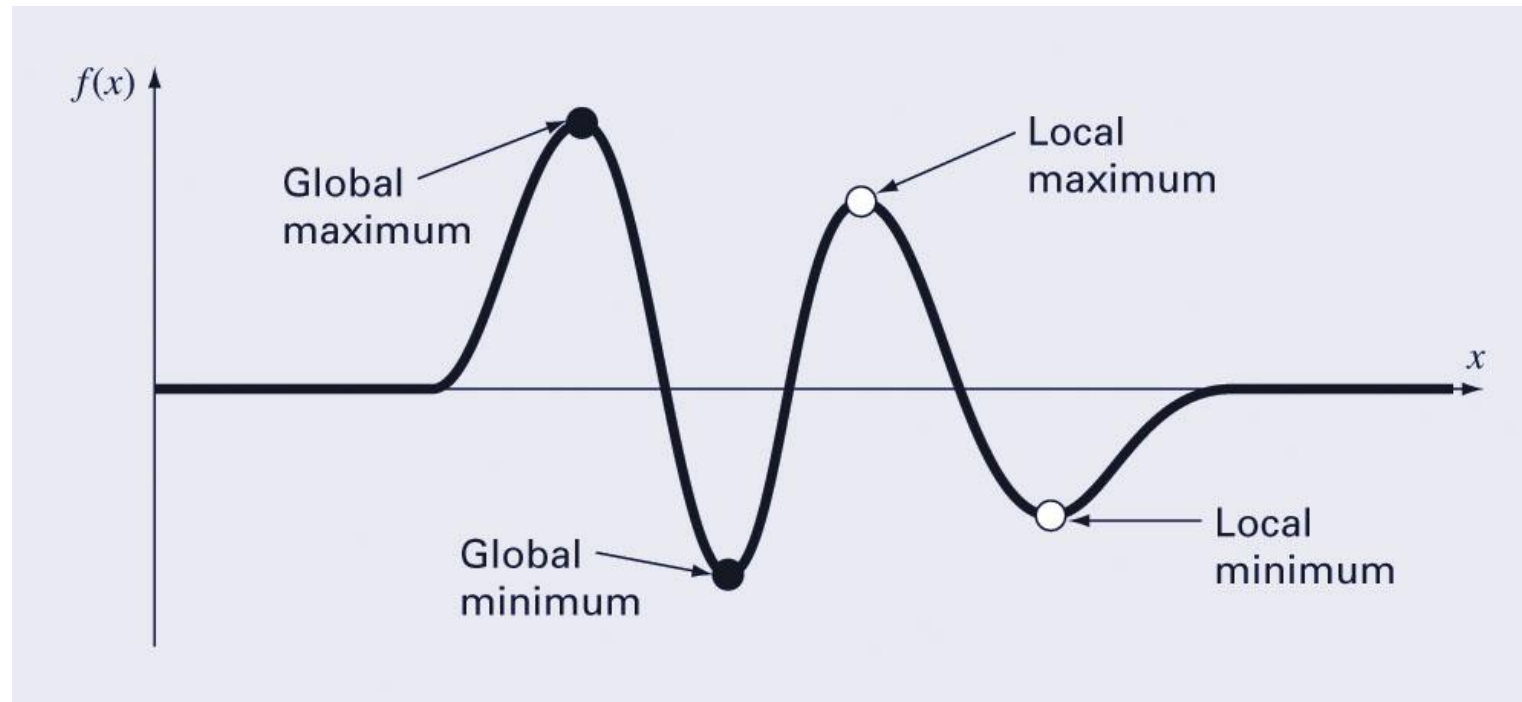
Summary

- Optimization review
- Evolutionary algorithms
- Genetic algorithm for MSA

Optimization Methods



Optimization Methods



Optimization Methods

- Exact solution
 - There is a practical algorithm for finding the optimum
 - E.g. dynamic programming for aligning two sequences
- Heuristic search
 - Trade off optimality for efficiency by taking short cuts
 - E.g. Multiple Sequence Alignment

Optimization Methods for MSA

- Simulated Annealing
 - Slow and works best with a solution that is already close to optimal
- Gibbs Sampling
 - Does not deal well with gaps
- Genetic Algorithms

Evolutionary Computation

- Evolutionary computation is a family of algorithms for global optimization **inspired** by biological evolution
- Use **populations of candidate solutions**, trial and error and selection to solve optimization problems.
- The best solutions, as defined by a **fitness test**, get to “reproduce” a new generation. Usually involving by stochastically (randomly) **removing poor fit solutions** and **introducing small random changes**.
- This is intended to simulate natural selection and mutations to create a population of solutions with the best fit for the problem.

Evolutionary Computation

- Evolutionary computation is a family of algorithms for global optimization inspired by biological evolution
 - Different from algorithm to simulate biological evolution
- Examples:
 - Particle Swarm Optimization
 - Genetic Programming
 - Genetic Algorithms

Genetic Algorithms

- Representation
 - Represent the "DNA" and genotype to phenotype mapping
- Fitness evaluation and selection
 - Score individuals and select which ones will "breed"
 - This uses the objective function
- Crossover and Mutation operators
 - Mix characteristics of parents
 - Introduce random changes

Genetic Algorithms

MSA with Genetic Algorithms

[Notredame, Cédric, and Desmond G. Higgins. "SAGA: sequence alignment by genetic algorithm." Nucleic acids research 24.8 \(1996\): 1515-1524.](#)

- Start with a population of random MSA:
 - Begin each sequence with 0 to $0.25 \cdot l$ gaps, at random, fill end with sufficient gaps, where l is the length of the sequences
- Score each alignment. The score will be used to obtain the fitness
 - If the score is a distance, then fitness can be $1/\text{score}$, for example
- Create next generation:
 - Populate 50% with the top 50% individuals from previous generation
 - The other 50% by crossing individuals selected at random with higher probability for those that are fitter

- Each P is a single MSA with the same sequences but gaps placed differently
- Each G is generation: a set of these MSAs
- First generation G_0 is randomly initialized
- Use scoring to select individuals – better score, more children, lower score are replaced by children
- Children are created by operators that have as input either 2 individuals (crossover) or 1 individual (mutation)

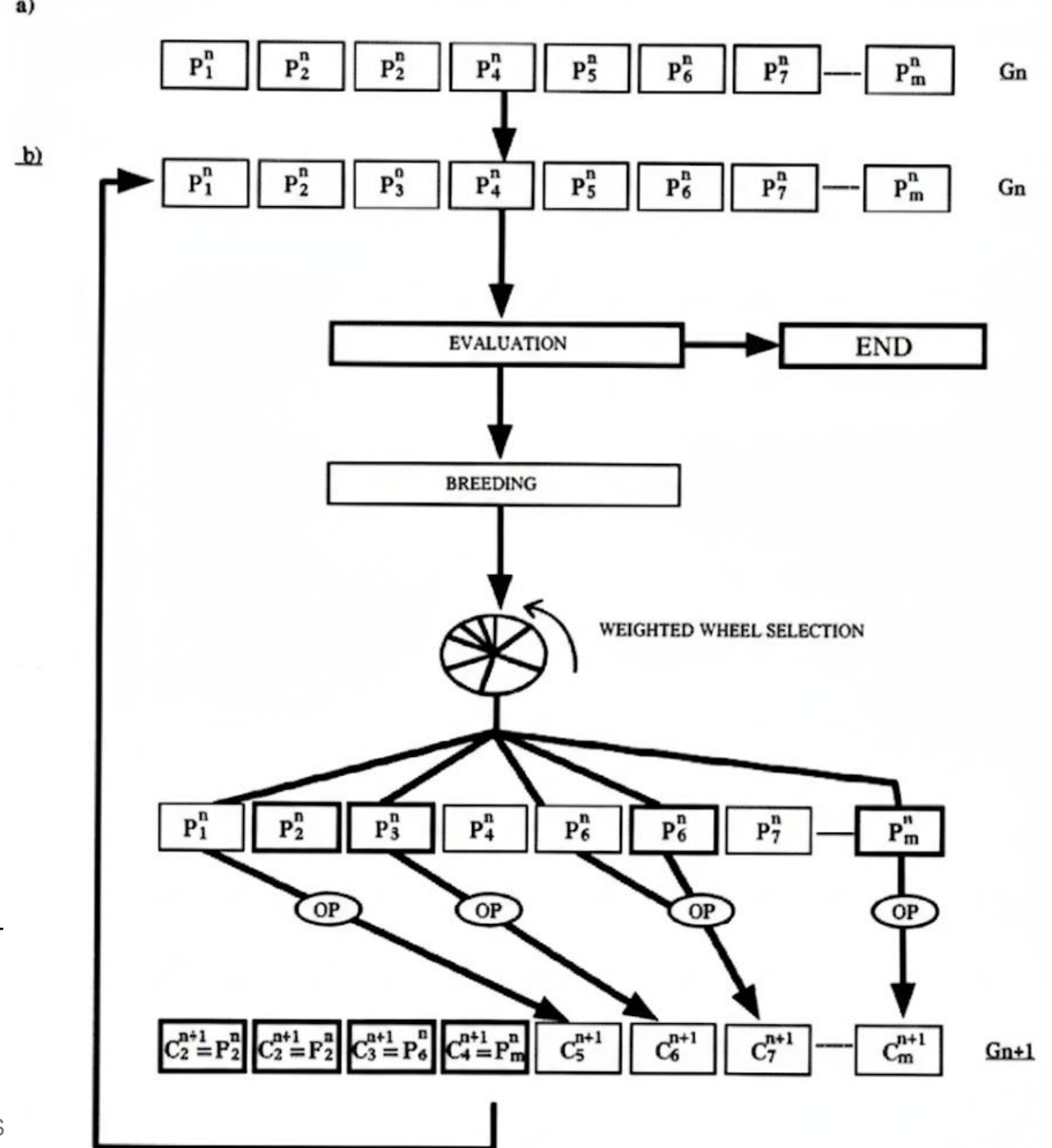
Initialisation 1. create G_0

Evaluation 2. evaluate the population of generation n (G_n)
 3. if the population is stabilised then END
 4. select the individuals to replace
 5. evaluate the expected offspring (EO)

Breeding 6. select the parent(s) from G_n
 7. select the operator
 8. generate the new child
 9. keep or discard the new child in G_{n+1}
 10. goto 6 until all the children have been successfully put into G_{n+1}
 11. $n = n+1$
 12. goto EVALUATION

End 13. end

MS



Crossover Operator

Require two parents

- Point crossover:

- Select a line between two columns on parent A
- Select equivalent split in parent B (may not be a line)

- Select best children

AC - T - CA	ACT - C - A
ACAT - GG	AC - AT GG
GC - T T - A	GCT TA - -

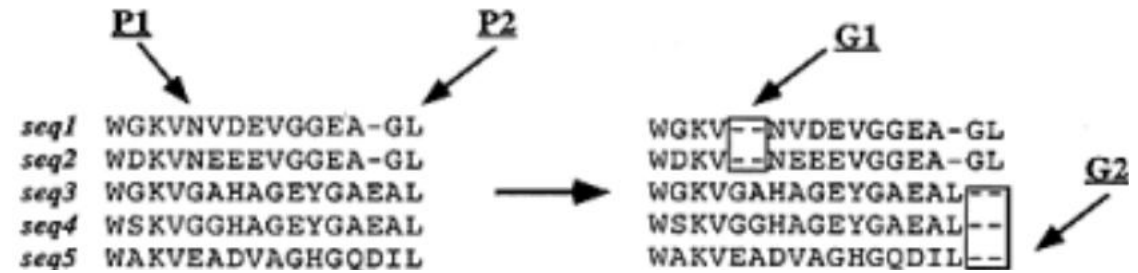
- Uniform crossover:

- Find columns that are identical in both parents.
- Keep those, swap blocks between the identical columns
- Harder to implement

Mutation Operators

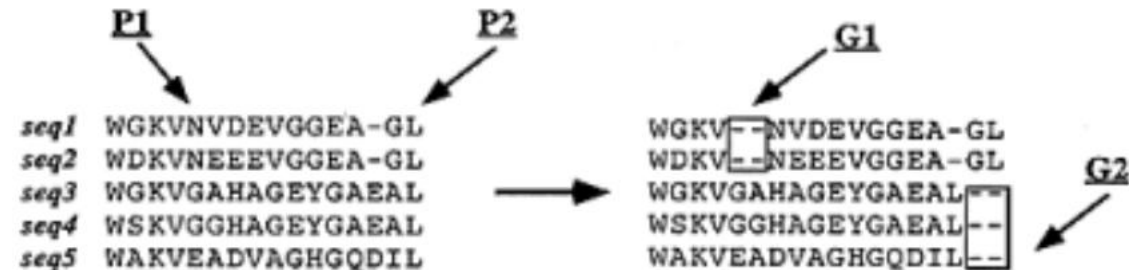
Applied to a single MSA

1. Gap insertion of random size at a random position on a single sequence
2. Deletion of a random gap block
3. Single gap insertion on randomly selected sequences
4. Move random gap block to the beginning of the sequence
5. Reduce size of random gap block by one
6. Increase size of random gap block by one
7. Move a gap block one position in either direction
8. Split a randomly selected gap block



Mutation Operators

Applied to a single MSA



9. Combine two adjacent gap blocks into either side
10. Swap two randomly selected gap blocks of the same sequence
11. Swap two randomly selected gap blocks from different sequences
12. Replace gap block ($l > 1$) with a single gap
13. Move one or more gap blocks a random number of positions
14. Delete all gap blocks of a single sequence
15. Delete gap blocks from multiple sequence at the same position
16. Move a gap block to another sequence

MSA with Genetic Algorithm

- General idea:
 - Use population of MSA
 - Score individuals, select best for next generation and breeding
 - According to the Fitness score/Expected Offspring:
 - Combine parts of parents
 - Insert random mutations
 - Repeat until stop criteria is achieved (e.g. number of iterations, score threshold, stops improving, ...)

MSA with Genetic Algorithm

- Important to remember:
 - Size of population should be constant
 - Sequences (including gap symbols) must have the same length
 - If you add a gap to a sequence, you increase the length of that sequence by one, should the length of all other sequences should be increased. Likewise if you delete a gap
 - Make sure the sequences themselves don't change: if the gaps are removed, the sequence of symbols each sequence should never change
 - No column can be all gaps – clean your alignments
 - No duplicates

Bibliography

- Cédric Notredame, Desmond G. Higgins, SAGA: Sequence Alignment by Genetic Algorithm, *Nucleic Acids Research*, Volume 24, Issue 8, 1 April 1996, Pages 1515–1524, <https://doi.org/10.1093/nar/24.8.1515>